Sleuth: A Switchable Dual-Mode Fuzzer to Investigate Bug Impacts Following a Single PoC

Haolai Wei
IIE, CAS†
Beijing, China
School of Cyber Security, UCAS‡
Beijing, China
weihaolai@iie.ac.cn

Liwei Chen*
IIE,CAS†
Beijing, China
School of Cyber Security, UCAS‡
Beijing, China
chenliwei@iie.ac.cn

Zhijie Zhang
IIE,CAS†
Beijing, China
School of Cyber Security, UCAS‡
Beijing, China
zhangzhijie@iie.ac.cn

Gang Shi
IIE,CAS†
Beijing, China
School of Cyber Security, UCAS‡
Beijing, China
shigang@iie.ac.cn

Dan Meng
IIE,CAS†
Beijing, China
School of Cyber Security, UCAS‡
Beijing, China
mengdan@iie.ac.cn

Abstract

A proof of concept (PoC) is essential for pinpointing a bug within software. However, relying on it alone for the timely and complete repair of bugs is insufficient due to underestimating the bug impacts. The bug impact reflects that a bug may be triggered at multiple positions following from the root cause, resulting in different bug types (e.g., use-after-free, heap-buffer-overflow). Current techniques discover bug impacts using fuzzing with a specific coverage-guided strategy: assigning more energy to seeds that cover the buggy code regions. This method can utilize a single PoC to generate multiple PoCs that contain different bug impacts in a short time. Unfortunately, we observe existing techniques are still unreliable, primarily due to their failure in balancing the time between in-depth and breadth exploration: (i) in-depth exploration for bug impacts behind crash regions and (ii) breadth exploration for bug impacts alongside unreached regions. Current techniques only focus on one exploration or conduct two explorations in separate stages leading to low accuracy and efficiency.

Considering the aforementioned problem, we propose Sleuth, an approach for automatically investigating bug impacts following a known single PoC to enhance bug fixing. We design Sleuth on two novel concepts: (i) a dual-mode exploration mechanism built on a fuzzer designed for efficient in-depth and breadth exploration. (ii) a dynamic switchable strategy connecting with the dual-mode exploration that facilitates the reliability of bug impact investigation. We evaluate Sleuth using 50 known CVEs, and the result of experiment shows that Sleuth can efficiently discover new bug impacts in 86% CVEs and find 1.5x more bug impacts than state-of-art tools. Furthermore, Sleuth successfully identifies 13 incomplete fixes using the generated new PoCs.

CCS Concepts

• **Security and privacy → Software security engineering**.

Keywords

Bug impact, Fuzzing, Patch testing

ACM Reference Format:
Haolai Wei, Liwei Chen, Zhijie Zhang, Gang Shi, and Dan Meng. 2024. Sleuth: A Switchable Dual-Mode Fuzzer to Investigate Bug Impacts Following a Single PoC. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24), September 16–20, 2024, Vienna, Austria.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3650212.3680316

1 Introduction

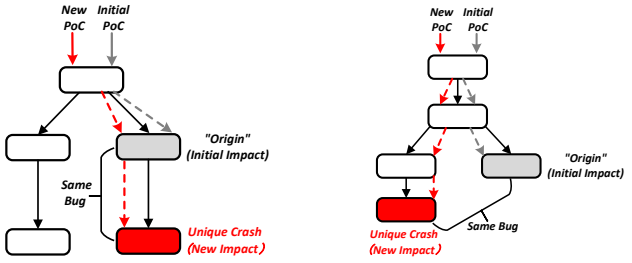
Fuzzing techniques [19, 26, 52] and various debugging/sanitization features [42, 53], which gained momentum among researchers, empowered the identification of numerous bugs in open-source software with ease. However, most bugs are reported with only a single proof-of-concept (PoC), which is insufficient for developers to analyze the severity of a bug, leading to the obstacle in prioritizing bug fixes [39, 49]. This often stems from a lack of clarity regarding the diverse ***bug impacts*** that a bug can produce. For instance, by following different paths or execution contexts, a bug that exhibits a null pointer dereference impact initially may also lead to an use-after-free impact. This may allow attackers not only to read illegal memory regions but also to construct malicious operations by altering control flow. As such, it could be misleading if developers only use a single PoC to infer the bug's severity.

From our perspective, bug impact reflects *the way in which attackers exploit a bug*. In the context of memory corruption bugs,

*Corresponding author: Liwei Chen (chenliwei@iie.ac.cn).
†Institute of Information Engineering, Chinese Academy of Sciences
‡University of Chinese Academy of Sciences

ISSTA '24, September 16–20, 2024, Vienna, Austria

ACM ISBN 979-8-4007-0612-7/24/09
https://doi.org/10.1145/3650212.3680316



(a) Behind Position. (b) Alongside Position.

Figure 1: New PoCs Trigger the Same Bug

once a bug is triggered, the state machine [25] of the victim program turns into a "weird machine" [24, 47], which allows attackers to perform unintended behaviors to construct exploits. We consider the bug impact can anticipate unintended behaviors of attackers and indicate possible risks. Generally, bug impact can be regarded as a measurable consequence when a bug is triggered following its root cause. In particular, we use a pair consisting of impact type (e.g., stack/heap-buffer-overflow, use-after-free) and crash position (code line where a crash occurs) reported by address sanitizer to represent the measurable consequence. If an input crashes with an unobserved "impact-pair", we consider it exposes a new bug impact. Exploring bug impacts provides developers with a convenient and efficient method to infer the severity of bugs, avoiding time-consuming and laborious root cause analysis [18].

A straightforward way to effectively explore bug impact is to **use a single PoC as an input and generate multiple PoCs from it through fuzzing**. This process emphasizes the fuzzer's attention on distinct paths that are directly related to the same identified bug. A typical implementation of this approach is AFL++'s "crash exploration mode"(*afl-cexp*) [51]. This mode harnesses AFL++'s established edge coverage feedback mechanisms, specifically orienting them to probe deeper into bug-related code regions. In addition, recent studies [33, 38, 54] have proposed effective strategies based on this concept. Evocatio [33] replaces edge coverage-guided with capability-guided, and it can uncover some new bug impacts by exploring the surrounding state space of the initial PoC. As for kernel bugs, GREBE [38] utilizes bug-related structure guiding fuzzer to explore the kernel bug impacts in various contexts. SyzScope [54] combines symbolic execution with fuzzing to deep into the code regions that are difficult to reach.

Although the existing studies have shown considerable strength in exploring bug impacts, we observe a critical limitation in previous work: they are inability to balance the time of in-depth and breadth exploration. Specifically, this limitation arises mainly because new PoCs trigger the same bug at two types of positions (Figure 1). Taking an initial PoC as a start, we regard its crashing position as an *"origin"*. (i) One position is located directly behind the *origin*. New PoC needs to pass through the *origin* and causes a new crash in the subsequent code region (Figure 1a). (ii) The other position is located alongside the *origin*, which may be far from the *origin* (Figure 1b). These two positions require distinct exploration approaches: An in-depth exploration to the covered code regions when focused on the former and a breadth exploration to unreached code regions for the latter. Furthermore, since exploration space can be vast, identifying which approach can efficiently discover more bug impacts is still an unresolved question. However, previous studies typically emphasize one exploration [26, 38] or conduct two explorations in separate stages [54], which in some cases confines exploration to only one position and causes a waste of resources, leading to a deficiency in the discovery of bug impacts.

Considering the aforementioned limitation, we propose Sleuth, a switchable dual-mode exploration technique to discover more bug impacts following a single PoC. Here, **dual-mode exploration** represents in-depth exploration and breadth exploration, and we design following mechanisms for these two modes respectively. (i) The in-depth mode is to find conditions for causing new bug impacts behind the *origin*. Previous work performs in-depth exploration by blindly changing inputs that reach the *origin*, which is highly time-consuming. In order to enhance efficiency, we introduce a new state tuple *"crash summaries"* for each crash position, and then we use this tuple to guide the fuzzer achieving in-depth exploration. (ii) The breadth mode focuses on unexplored code regions and it pre-identifies code regions that contain potential new bug impacts as exploration candidates. However, previous studies only identify these potential regions without assigning priorities, which may lead to misunderstanding the importance of a code region, wasting fuzzing energy. Consequently, we construct a *memory-relevant graph* to segment the priority of potential code regions and prioritize the exploration towards more promising regions. Finally, to enhance the flexibility of exploration and cover more bug impacts, we design a **dynamic switchable strategy** by monitoring some specific metrics to effectively switch between the two modes automatically. In brief, the inspiration behind our approach is to use a clue (a single PoC) to track evidence (bug impacts), ultimately revealing the truth (complete patch), much like a sleuth.

Our Sleuth is implemented on the basis of AFL++ and focuses on user-space memory corruption bugs. We collect 50 real-world CVEs to compare Sleuth with related advanced tools *afl-cexp* and Evocatio. The evaluation results show that Sleuth efficiently discovers a total of 856 impacts, exceeding afl-cexp's 584 and Evocatio's 202. and it discovers new bug impacts in 43 of the 50 CVEs. Besides, we conducted a bug severity assessment for each CVE using the impacts generated by Sleuth, and also discovered 13 CVEs that were not incompletely fixed.

We summarize the contributions of this paper as follows.

- We designed a novel approach named Sleuth for investigating bug impacts by generating multiple PoCs from a known PoC. Our goal is to provide more information for inferring bug severity and assist in prioritizing bug fixes.
- Sleuth utilizes a switchable dual-mode exploration to mitigate the challenge of balancing the time of in-depth and breadth exploration, and it also introduces flexible data flow coverage and priority strategy for each exploration mode.
- Results of experiment on 50 CVEs demonstrate that Sleuth outperforms the state-of-art bug impacts exploration tool in efficiency and practicality.

2 Background And Motivation

The following sections first detail bug impacts (Section 2.1) and provide a motivating example to reveal our concern (Section 2.2).

2.1 Elaboration on Bug Impact

As pointed out in previous work [25, 46], given multiple PoCs for a bug, the victim program could be turned into different *weird machine* states, which can be programmed by attackers to perform unintended behaviors in exploitation. In general, through which way to program these weird states depends on the impacts a bug exposes. For instance, an overflow reading of a heap buffer might allow an attacker to access sensitive data in memory or cause a denial of service on the system. Similarly, use-after-free in heap may lead to writing unintended values to freed objects (e.g., function pointers), which allows attackers to achieve privilege escalation by hijacking the control flow. In summary, impact reflects *the way in which attackers exploit the bug*.

Furthermore, some work introduces "bug capability" to measure bug severity, reflecting *what exactly attackers can do when exploiting the bug*, which naturally includes the bug impact. For example, the bug involves a heap-buffer-overflow impact in the previous paragraph, and through bug capabilities, one can further understand how many bytes can be accessed to the victim buffer, and how far the access can reach. Obviously, exploring the complete impact a bug exposes is a premise for enriching bug capabilities.

In reality, bug severity is reflected in the extent to which attackers exploit a bug to damage the system. According to the specific security requirements, every bug impact has its own severity attribute [45]. For instance, if maintaining the system's normal operation is critical, then a bug with null pointer dereference or out-of-bounds read is more severe. However, if the intention is to control the system, bugs capable of performing arbitrary writes become the primary focus. Additionally, we consider the position where the security violation happens as part of the impact. In the case of the same impact type, different positions may involve unique behaviors or objects that attackers can exploit [22]. Following, we elucidate the components of bug impact in this paper:

Impact type. Different types observed when triggering the same bug (e.g., heap-buffer-overflow, heap-use-after-free). We also added the type of memory access (read/write) that occurs when a crash happens. Bugs with more impact types can enrich attackers' exploitation ways, making exploitation easier to accomplish.

Crash position. The code line where the crash occurs when triggering a bug (e.g., *Line 17* in Listing 1). More crash positions imply that attackers have a larger space to achieve exploitation.

Finally, the "impact-pair" is <*impact type*, *crash position*>. Anyway, the more impacts a bug has, the more opportunities it has to cause damage to the system, making the bug more severe.

2.2 Motivating Example

We modeled on a real-world example[1] in Listing 1 to illustrate the motivation of our work. This program contains a heap overflow vulnerability due to a lack of boundary checks, which may crash at *Line* 17 depending on the values of *m_size* and *col*. Attackers could cause the program to crash at different positions by developing PoCs that execute different paths.

Specifically, the program allocates a memory of size *m_size* at *Line* 3 in function *extractA*, and then calls the functions *extractB* and *extractC* separately through different conditions to perform file information extraction. Once the value of *col* in the loop process goes beyond the boundary of the memory *dst* and meets the condition *cond*, heap buffer will be triggered at *Line* 17 and 29. Furthermore, if the value of *col* is equal to *m_size* − 1 in function *extractC*, the program will also crash at *Line* 20.

Listing 1: A Real-world Example

```
1   int extractA(uint32 n, uint32 s, uint32 e){
2       uint8* out = (uint8 *)malloc(m_size);
3       switch(n){
4           //Divide into two code regions.
5           case EXTRACT_1:
6               extractB(out + offset_1, s, e);
7           case EXTRACT_2:
8               extractC(out + offset_2, s, e);
9       }
10  }
11
12  int extractB(uint8* in, uint32 st, uint32 en){
13      uint8* dst = in;
14      col = en - st;
15      if (cond){
16          //Origin crash if col > m_size.
17          buf = *(dst + col);
18          ...
19          //Behind position when col = m_size - 1.
20          *(dst + col + 1) = bytebuf;
21      }
22  }
23
24  int extractC(uint8* in, uint32 st, uint32 en){
25      uint8* dst = in;
26      col = en - st;
27      if (cond){
28          //Alongside position crash similar to Line 17.
29          buf = *(dst + col);
30      }
31  }
```

Chanllenge. Prior experiments [47, 54] have indicated that simultaneously exploring the crash positions at *Line* 20 and *Line* 29 is a challenging task for fuzzing. On the one hand, reaching *Line* 20 requires modifying the control flow and data flow to the origin, which demands an enormous exploration space. Worse yet, it is unclear whether *Line* 20 can actually be triggered, and it slows down the exploration process of *Line* 29. On the other hand, triggering *Line* 29 requires exploring different path coverage, which results in insensitivity to data flow, and it also leads to insufficient exploration of reached code regions. In summary, the most important challenge is to balance the exploration towards to the two different positions of bug impacts, which we refer to as in-depth exploration and breadth exploration.

Solution & Goal. Recognize the above-mentioned issues, we aim to adopt a strategy that automatically switches exploration modes based on the feedback information during fuzzing. If the current exploration mode loses its *"potential"* to discover new bug impacts, we smoothly switch to another exploration mode. In this way, we can effectively alleviate the time waste caused by being obsessed with exploring one type of position, thereby discovering more bug impacts. It should be noted that our goal is to identify as many bug impacts as possible rather than all of them.

3 Design & Overview

Our overall design is primarily aimed at answering this question: *Given a known bug's PoC, how to effectively balance in-depth and breadth exploration to discover various impacts of this bug?* The method is to automatically determine the exploration mode based on the feedback generated during fuzzing. For this purpose, we set two types of feedback information: One is *crash summary*, a tuple containing the data flow state of a crash position, which is extracted from a crash report. Another is *memory-relevant graph*, whose

[1] Simplified CVE-2023-0799

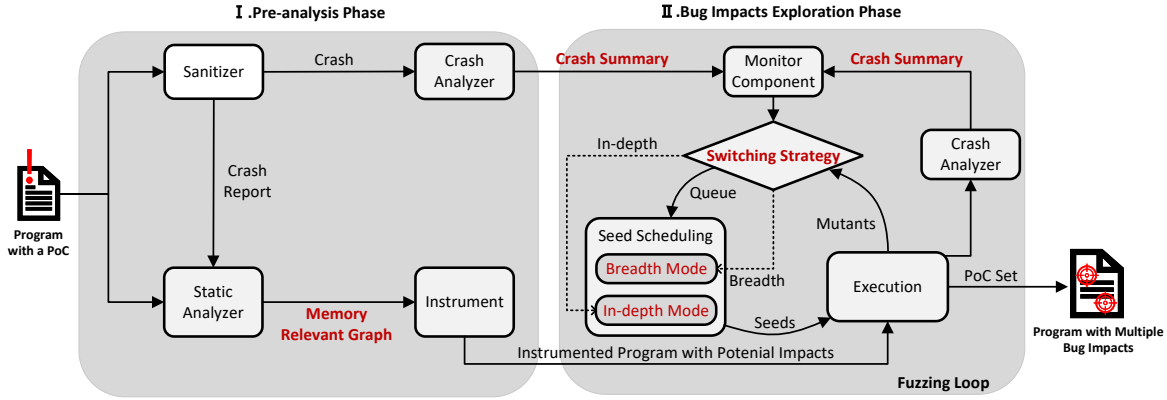


Figure 2: Overview of Sleuth's Workflow

nodes represent the potential bug impacts. Our idea for achieving automatic switching is that once new crash summaries appear during fuzzing, we prioritize in-depth exploration for explored code regions. Otherwise, we use the feedback of the instrumented memory-relevant graph to conduct breadth exploration.

From this perspective, we design our approach Sleuth through two phases: *pre-analysis phase* and *bug impacts exploration phase*. We display the overview of Sleuth's workflow in Figure 2 and then provide a brief overview of our design.

Pre-analysis Phase. In this phase, Sleuth takes in as input a program with a single PoC and provides the instrumentation of required information for the dual-mode exploration. Here mainly consists of three tasks:

- Extracting crash summaries using a crash analyzer.
- Constructing the memory-relevant graph using a static analyzer which consists of taint analysis and a level distribution strategy.
- Instrumenting the above information into the program.

Bug Impacts Exploration Phase. In this phase Sleuth rescopes traditional coverage-guided fuzzers by utilizing the switchable dual-mode fuzzer to generate multiple bug impacts. The fuzzer introduces the following new strategies:

- A switching strategy combined with a monitor chooses the exploration mode to switch.
- The dual-mode exploration strategy in seed scheduling to select promising seeds.

Finally, We extract new bug impacts from the set of new PoCs. Additionally, our approach is still founded on coverage-guided fuzzing and is similar to directed fuzzing [19, 23, 31]. However, we did not directly replace the code coverage guidance with target guidance strategies, because a standalone targeted method makes it difficult to vary the paths after reaching the crash positions.

4 Approach

In this section, we elaborate on the design and technical details of Sleuth. At a high level, Sleuth is a tool consisting of a static analyzer and a fuzzer with a crash analyzer that takes in as input a program and a PoC, and returns a set of PoCs involving various bug impacts.

4.1 Pre-Analysis Phase

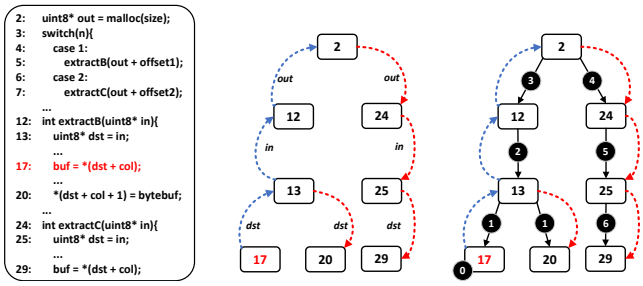
Here, we explain how crash analyzer and static analyzer operate, as well as the process of instrumenting potential bug impacts into the program.

4.1.1 Crash Analyzer. When using the crash exploration mode of fuzzing, fuzzers can generate inputs that reach the crash positions via different paths, which helps discover more impacts of a single bug. Unfortunately, this process is blind and inefficient. As we mentioned in Section 2.2, the "*potential*" of any crash position is unknown, so in the absence of guidance, fuzzers cannot determine which positions are worth exploring in-depth, and may move exploration away from positions that can truly expose impacts. To meet this requirement, we propose *crash summaries* (inspired by Evocatio [33]) to monitor the explorable potential of each crash position during exploration.

The crash summary in this paper represents a special crash of the tested bug, which is defined as a tuple $< P, (O, N) >$, including three critical elements: the **crash position** P, the **offset within the victim object** O, and the **number of bytes accessed** N. For example, the crash is triggered at *Line* 17 in Listing 1 with an overflow buffer. Assuming the boundary address of the buffer is 10, and the accessed address through the crash is 15, with the ability to read 3 bytes. Then the value of P is *Line* 17, the O is 5, and the N is 3. Usually, one P may correspond to multiple pairs of (O, N).

The crash summary is used to uniquely identify a crash and assists exploration mode switch stem from two concepts: First, the P records the position where a crash occurs, and it constrains the control flow that triggers this crash. Second, the (O, N) reflect the illegal behaviors at the crash position, and they constrain the data flows that pass through this position. In this way, when P remains constant but (O, N) change, we perform in-depth exploration for the same crash position; when there are no signs of change in (O, N) of the P, we guide P to change and perform breadth exploration for unreached code regions.

We capture crash summaries through the built-in functions of sanitizer [4]. In detail, we extract stack frames from ASAN's backtrace, which records the call stack when a program crashes, and calculate the hash of these stack frames as the value of P. Then we match the crash information captured by ASAN to obtain the values



(a) Code Snippet. (b) Taint Analysis. (c) Level Distribution.

Figure 3: An example of generating *MRG* through the static analyzer. Assume *Line 17* is the initial crash position and pointer *dst* points to the victim object.

of O and N. In addition, we also enable the program to continue execution after a crash by disabling the *halt_on_error* option. This is because sometimes a crash position P corresponds to only one pair(O, N), causing it impossible to observe data changes. Thus we enable continued execution to supplement more crash summaries for an input. It should be noted that other crash features can also be extracted from crash reports, including the value of program counter, access type, etc. These features have little relevance to in-depth exploration, so we only record them and do not include them in the crash summary. Furthermore, the crash summary is only applicable to out-of-bounds bugs, as some memory corruption bugs do not result in illegal address access. In such cases, we only record their crash positions as crash summary.

4.1.2 Static Analyzer. The memory corruption bug occurs as a result of illegal access to a memory object (e.g., accessing a buffer with an unchecked index) or performing illegal usage (e.g., dereferencing a free pointer). In general, there may be multiple instructions that operate this memory object, and these instructions still carry potential risks of triggering the bug. Besides, this memory object may have relevant objects (e.g., the pointer *dst* in motivating example 1), which have the point-to information passed from initial object and can also access the buggy memory. In this paper, we consider that the new impacts caused by the victim object and its relevant objects belong to the same bug. These objects may occupy the same or similar memory regions as the victim objects, which means they are prone to cause the same bug following the root cause. We regard the instructions that access the above-mentioned objects as the target with potential bug impacts and construct the memory-relevant graph to assist in exploration. Our static analyzer consists of taint analysis and a level distribution strategy.

Taint Analysis. This static analysis technique is theoretically sound and can purposely track target objects with point-to relationships [15]. Therefore, we can efficiently cover most possible positions that may contain new bug impacts.

In this process, Sleuth employs backward and forward taint analysis to find relevant memory objects. (1) First, we obtain the instruction where the crash occurs from the sanitizer and identify the operated variables in this instruction. Then, Sleuth takes these variables as *source* sites and tracks them backward following the data flow to the definition sites. The defined variables corresponding to these sites are pointing to relevant memory objects. Besides, we filter these variables based on types and only retain variables prone to memory bugs, including arrays, pointers, and structures. (2) Second, we take the collected defined sites as *sources*, and track forward to the *sink* sites where variables are accessed (read or write). This step is similar to generating *def-use* chains. It should be noted that new definition sites may be discovered during the forward analysis. We also add these sites to the *source* collection and track the *sink* sites until no new nodes are discovered. Finally, we collect all *sink* sites as the set of memory-relevant nodes.

For all steps of taint analysis, we use the code snippet in Figure 3a as an example. We first extract the initial crash position at *Line* 17 and identify the pointer *dst* points to the victim object. Then, we take the pointer *dst* as the *source* site and track backward to *Lines* 13, 12, and 2 where definitions were made. The defined variables *dst*, *in*, and *out* point to the relevant memory objects (Figure 3b). Finally, we perform forward analysis and identify *Line* 20 and *Line* 29 which may contain potential impacts.

Level Distribution. We have collected potential positions containing impacts through taint analysis. However, exploring all these positions is inefficient, and we need to prioritize more promising ones. For memory bugs, relevant objects that are *"closer"* to the initial victim object are likely to cause new crashes. We use the number of interval data flow edges to measure the *"closer"*. Intuitively, if value flows exist between the definition sites of two pointers, they also convey point-to information. We refer to these two pointers as *source pointer* and *destination pointer*. However, the point-to information passed from the source pointer may have address offset, only allocating the portion of memory it points to the destination pointer (e.g., *Lines 5 and 7* in Figure 3a). In that case, the more times point-to information transmissions occur, the less similar the memory regions they occupy, which means fewer chances to trigger the bug.

According to this observation, we propose a level metric L to measure the score of *"closer"*. We perform the level distribution process after taint analysis and construct a memory-relevant graph (***MRG***) that only contains *def* and *use* sites of memory-relevant objects. Since an object may correspond to several *def* sites, the data flows reaching the object during program execution are various. It is inappropriate to sign an object with a fixed value. Therefore, we calculate the level of each edge in *MRG*, and we denote the set of nodes in *MRG* as N and the set of edges as E. Each edge $e(e \in E)$ can be uniquely identified as $n_j \hookrightarrow n_i$, indicating the node n_j is data dependency [37] on the node n_i, where $n_j, n_i \in N$ and $i \neq j$. Intuitively, we denote the initial bug object as t, serving as the start node of *MRG*, and set up its level as 0. Our level calculation is as follows:

$$L(e) = \begin{cases} 1 & e.n_j = t \\ 1 & e.n_i \vDash t \hookrightarrow n_i \\ min(L(e_{pre})) + 1 & \text{other causes,} \end{cases} \quad (1)$$

where $e \in E$ and $n_i, n_j \in N$. We set the level of the edges flowing into the start node as 1. Beyond that, if the node coincides with the *def* site of the start node, we also set the level of edge flowing into it to 1. As for other edges, we calculate the level for each edge by adding 1 to the level of its precursor edge e_{pre}. The precursor

edge represents the edge that flows into the *def* site of the current edge. Since there may be multiple precursor edges, we take the one with the minimum level for calculation. We apply the breadth-first algorithm when traversing the *MRG*, prioritizing the level distribution to edges that link to visited nodes.

We illustrate the level calculation in Figure 3c. *Line 17* is the start node and the number inside the solid circle represents the level. Except for *Line* 20 coincided with the *def* site of the start node, the level of other edges increased with the number of data flow edges. Finally, we construct the memory-relevant graph. In this graph, each node represents an operation on a relevant object which may contain the potential impact, and each edge represents the data flow with the level of relevance. Given this memory-relevant graph, we can guide fuzzing to explore more promising code regions to find new bug impacts.

4.1.3 Instrumentation. Alongside the generation of memory relevant graph, we instrument the information in this graph as coverage metrics into the program, guiding fuzzing to efficiently explore bug impacts during breadth exploration mode. Since the nodes in memory-relevant graph may be located in two non-adjacent basic blocks, it is not feasible to represent the edges by recording the previous and current basic blocks like traditional methods [51]. For this purpose, we design a new algorithm for instrumentation and show it in Algorithm 1.

Algorithm 1: Memory-Relevant Instrumentation

```
Input: target program P, and memory-relevant graph G
Output: instrumented program P
BB ← ∅
Level ← {}
InFlow ← {}
E ← get_edges(G)
for e ∈ E do
    insert Level[e] ← get_level(e, G)
    bb_end, bb_start ← get_basic_block(e)
    BB ← BB ∪ bb_end ∪ bb_start
    insert InFlow[bb_end] ← bb_start
for b ∈ InFlow[i], 0 ⩽ i < len(InFlow) do
    P ← instrument_global(b, s, 0)
for f ∈ P do
    for b ∈ f do
        if b ∈ BB then
            P ← instrument_tag(b, random())
    for b ∈ f do
        if b ∈ InFlow then
            P ← instrument_cov(
            tag(b), tag(InFlow[b]), Level)
return P
```

First, we prepare the information required for the instrument. For each edge e in memory relevant graph G, it flows from the starting node into the ending node. Sleuth uses bb_{start} and bb_{end} to represent the basic blocks where these two nodes are located, and it collects all basic blocks into *BB*. Besides, Sleuth uses a map *InFlow* to collect the set of bb_{start} for each bb_{end}. If bb_{start} is same as bb_{end}, Sleuth do not reserves the bb_{start}.

Second, the instrumentation process is simplified at *Lines* 10-19, and we mainly design three phases: (1) for each bb_{start}, Sleuth instruments a global variable s in the program by traversing *InFlow* and assigns 0 to the variable; (2) for each basic block b in the program, if it exists in *BB*, Sleuth instruments a random *tag* at entry to uniquely mark this basic block. (3) Sleuth finally uses *instrument_cov()* to instrument edge and level at each bb_{end}.

In detail, assuming $bb^i_{end} \hookrightarrow < bb^i_{start\_1}, bb^i_{start\_2}, \ldots, bb^i_{start\_n} >$, we design the calculation of coverage instrumentation *cov* for edges at each bb_{end} is:

$$\begin{aligned} & tag(bb^i_{end}) \oplus (tag(bb^i_{start\_1}) \wedge bb^i_{start\_1} \rightarrow s) \\ & \qquad \ldots \\ & \qquad \oplus (tag(bb^i_{start\_n}) \wedge bb^i_{start\_n} \rightarrow s), \end{aligned} \tag{2}$$

where $0 \leqslant i < len(InFlow)$, $0 \leqslant n < len(InFlow[bb^i_{end}])$. When a bb_{start} is covered during program execution, we set its global variable s to 1. In this way, we can identify that bb_{end} is reached from which bb_{start}, thereby determining the edges covered in the memory-relevant graph. As for the level of each edge, due to the presence of multiple edges participating in the calculation, we take the minimum value among the covered edges. In addition, we also assign a fixed value *idx* for each bb_{end} to record the covered basic blocks. Each edge of the memory-relevant graph is stored as a key in a new shared memory, with its level and *idx* as the value. We define each entry of the new shared memory as:

$$\_\_afl\_mrg\_map[cov] = (idx << 16) \mid level, \tag{3}$$

where we set 32 bits to store value for each entry. The most significant 16 bits are used to store *idx*, and the remaining bits store the level. As we need to explore unreached code regions, we still retain traditional control flow edge coverage instrumentation and utilize it as a supporting feedback metric.

4.2 Bug Impacts Exploration

In this phase, we introduce the design of exploration mode switching strategy and the corresponding optimization for fuzzing tool.

4.2.1 Switching Strategy. In order to enhance exploration efficiency and discover more bug impacts, we introduce a *"monitor"* to conduct the switch of in-depth and breadth exploration. This monitor primarily records the occurrence of new crash summaries and checks the output of execution at regular intervals. In this strategy, we follow the principle of in-depth-first exploration because it is more likely to encounter new crashes near the already crashed positions.

Associated with the crash summary $< P, (O, N) >$, the monitor maintains two metrics τ and ε. 1) τ is the time interval from the last discovery of new crash position P to the present, and 2) ε is the execution interval between the last discovery of the pair (O, N) and the present. Sleuth prioritizes conducting in-depth exploration at the beginning of fuzzing. In this mode, we declare a boolean variable $S_{d \rightarrow b}$ to represent the switch from in-depth to breadth and construct the ξ to represent the set of discovered crash summaries. We show the update conditions of $S_{d \rightarrow b}$ are as follows:

$$S_{d\to b} = \begin{cases} 1 & P \in \xi \wedge \tau > maxTime \\ 1 & (O,N) \in \xi \wedge \varepsilon > maxExec \\ 0 & other\ cases, \end{cases} \quad (4)$$

where we set the maximum interval time $maxTime$ and maximum interval execution $maxExec$. If the monitored metrics are greater than the threshold and no new crash summaries are detected, Sleuth will switch to breadth exploration mode. It should be noted that the purpose of setting thresholds is to improve exploration efficiency. Therefore, we set the time threshold to prevent long periods of unproductive in-depth exploration and the execution threshold to eliminate crash positions with no exploration value. We temporarily set $maxTime$ to 8 minutes and $maxExec$ to 50000 counts.

As for the breadth exploration mode, once new crash positions P appear, Sleuth switches to the in-depth exploration immediately. After switching the exploration mode, we select priority seeds for the next execution based on the current exploration mode. We mainly introduce the mechanisms of each exploration mode in the following section.

4.2.2 Dual-Mode Exploration. For the efficient operation of each exploration mode, we design unique seed retention and seed selection algorithms for in-depth and breadth exploration. We introduce the details in the following section.

In-depth Exploration. In this mode, Sleuth selects seeds to break through the crashes and explore deeper code regions. Specifically, Sleuth only retains crashing test cases that introduce new crash summaries. These test cases are stored in the fuzzer queue with other seeds to be mutated and help guide the fuzzer towards undiscovered crash summaries. We use an additional hash table to store and check crash summaries.

As for seed selection, since seeds in the queue may generate new crash summaries in multiple code regions, we select "favored" seeds based on whether they reach unseen crash positions, which are more promising. For this reason, we introduce a set to store the discovered crash positions. First, if the crash position reached by a seed is not in the set, Sleuth picks up this seed as the preference. Second, if no new crash position is found, Sleuth identifies the seed with the fastest speed and perfers that one instead.

Breadth Exploration. In this mode, Sleuth selects seeds to maximize the number of potential bug impacts triggered in unexplored code regions and follows the strategy of coverage-guided fuzzers. Accompanied by the instrumented potential bug impact using *MRG*, Sleuth includes the mutated seed into the seed queue only if one of the following two conditions holds: First, the seed reaches out to an unseen basic block involving potential bug impacts. Second, the seed covers new edges and reaches discovered basic blocks that contain potential bug impacts. By following these two conditions, we can ensure reaching the intended code regions while exploring different paths to trigger potential bug impacts.

As not all instrumented basic blocks can trigger crashes successfully, we prioritize selecting seeds for mutation that contain memory objects most relevant to the bug. With the instrumentation in Section 4.1.3, Sleuth determines the relevance through the level of basic blocks. Specifically, for all the seeds in the queue, Sleuth sorts them based on the level of new basic blocks they pass through. First, we set up a set for newly reached basic blocks, and assign the level to each basic block based on instrumentation. Sleuth records the new basic block with the highest level and selects the seeds in the queue that pass through this basic block as "favored". Second, if no new basic blocks are discovered, Sleuth follows the strategy of a traditional coverage-guided fuzzer and prioritizes selecting seeds that discover new edges.

We design the above rules based on the characteristics of each exploration mode and generate seeds that are more promising to discover new bug impacts. Sleuth applies these rules to streamline the seed queue and then performs seed scheduling for energy assignment.

5 Implementation

Sleuth is based on the LLVM 12.0.0 [35] infrastructure and the fuzzing tool AFL++, and we implement the switchable dual-mode of Sleuth using 6k lines of C/C++ code. We will describe some details of our implementation.

During the phase of Pre-analysis, inspired by Evocatio [33], we modified the original sanitizer using the interfaces in *asan_interface* and *execinfo* libraries to synthesize the crash analyzer. As for the static analyzer, we input a single PoC and LLVM IR of the target program. In our implementation, we extracted the initial crash position and the corresponding instruction from the ASAN report. Then, based on the function of prior work [30], we utilized SVF's Andersen pointer analysis [15, 44] to build SVFG, and performed taint analysis in SVFG to collect memory-relevant objects. Briefly, we integrated all the analysis functions into a single program and wrapped them within the LLVM compiler. Finally, we ultimately generated a JSON file to save the memory-relevant graph.

During the phase of instrumentation and bug impact exploration, as is described in Section 4.2, we performed our instrumentation by developing Clang passes and declaring a new bitmap with a size of 64KB which has 2^{14} entries and each entry occupies 4 bytes. We modified the seed selection and queue trimming strategy based on AFL++. Additionally, we employ several Python scripts around 1K Lines to facilitate the extraction of information from crash reports and the analysis of experimental data.

6 Evaluation

We evaluate Sleuth by answering the following research questions:

RQ1: Does Sleuth discover a wider range of bug impacts (compared to other bug impact exploration tools)?

RQ2: Does Sleuth discover new bug impacts more effectively (Less time spent compared to other tools)?

RQ3: Can Sleuth be applied to real-world bug patch testing?

We design experiments for each question and utilize study cases to illustrate the practical significance of Sleuth.

6.1 Experiment Setup

Evaluation Benchmark. In Table 1, we collected 50 CVEs from 10 open-source programs that were disclosed within the past five years. We selected 25 CVEs from Evocatio's benchmark, excluding those that are relatively old or cannot be reproduced due to the lack of PoC. In addition, we added 25 CVEs on this basis, which contain a richer exploration space for bug impacts. According to the ASAN

report, we defined 9 impact types and listed the initial impacts of CVEs contained in each program.

In addition, to evaluate the effectiveness of Sleuth after eliminating false positives, we collect "completely fixed" CVEs which imply all PoCs generated by Sleuth do not cause crashes in the patched version for corresponding CVEs. We marked the 21 CVEs with symbol † in Table 2.

Table 1: Benchmark Summary. The impact types are: heap-buffer-overflow read/write (HOR/HOW), use-after-free read-/write (UAR/UAW), global-buffer-overflow read/write (GOR/-GOW), stack-buffer-overflow (SOF), null-pointer-dereference (#N) and wild-address-read (#W).

Program	Description	LOC	Type	#Quantity
Libtiff	Image library	84k	HOR, HOW, GOW, #W	9
Libming	Flash library	83k	HOR, HOW, UOR, #N	15
Binutils	Binary tools	2.1M	HOR, HOW, SOF, #W	10
Libsixel	Image library	37K	HOW	2
Jasper	Image manipulation	53K	HOR, HOW	4
Libsndfile	Audio manipulation	85K	HOW	1
Fig2dev	Graphics creation	47K	SOF	2
Yasm	Assembler	67k	UOR, SOF,#W, #N	5
Liblouis	Translator library	53k	GOR	1
Cflow	Source Code Analyzer	37k	UAR	1

Baseline Fuzzers to Compare. We compare Sleuth with two state-of-the-art fuzzers, AFL++ and Evocatio. AFL++ is currently the most widely-used fuzzing tool which is applicable in various scenarios. We utilize the AFL++'s "crash exploration mode"(*afl-cexp*). Evocatio is a new capability-guided fuzzer, that utilizing an initial PoC to explore the surrounding state space for new capabilities. Although Evocatio mainly explores the same code region, we compare it to Sleuth to demonstrate that we have included most bug impacts it discovered.

Performance Metrics. We evaluate performance by considering (i) the number of new bug impacts discovered over time, and (ii) the efficiency of finding these bug impacts.

Configuration Parameters. Since Sleuth requires a known PoC as input, it is necessary to add the *-C* option as runtime, following the configuration in *afl-cexp*. Taking into account the randomness introduced by the seed mutation of fuzzing, we set the duration of each fuzzing session to 12 hours and perform each experiment for 5 rounds.

Experiment Infrastructure. All our experiments have been performed on machines with an Intel(R) Xeon(R) Gold 5115 CPU (2.40GHZ) with 80 cores and 64GB of RAM under 64-bit Ubuntu 18.04 LTS server.

6.2 RQ1:Bug Impacts Discovery

Table 2 summarizes the bug impacts discovered across our benchmark (the first 25 in this table are from Evocatio's benchmark, and the subsequent 25 are test cases we added). Here, we focus on two metrics, one is the number of switches (**#Switch**) after exploration, and the other is the number of new bug impacts (**#Impacts**). Besides, we count the types of new bug impacts (**#Type**). For example, assuming the newly discovered bug impacts of a CVE are: <HOW, a.c:20>, <HOW, a.c:28>, <HOR, a.c:30>, then the **#Impact/#Type** is **3/2**. This can further reflect the diversity of impacts discovered by different fuzzers.

Table 2: Impacts discovered results by Sleuth, afl-cexp and Evocatio. "Type" records the count of new impact types. The average count of switches is also provided. "Increase" represents the impact increase rate, computed as (Sleuth - fuzzer) / fuzzer. The CVEs marked with symbol † are "completely fixed CVEs".

CVE ID	#Switch	#Impact / #Type			
		Sleuth	afl-cexp	Evocatio	Total
CVE-2018-17795†	2	2/1	1/1	1/1	2/1
CVE-2018-12900†	4.4	7/4	3/3	5/5	8/5
CVE-2018-8905†	2	0	0	0	0
CVE-2020-11895	109	65/6	44/6	22/5	68/6
CVE-2020-11894	59	58/5	44/5	16/5	64/5
CVE-2020-6628	86.2	58/6	43/5	4/2	63/6
CVE-2019-16705	36	30/6	31/6	10/4	42/6
CVE-2018-20591	31.4	65/6	38/6	13/5	72/6
CVE-2018-9009	84.3	42/5	39/5	8/3	46/5
CVE-2018-8964	67	40/5	38/5	19/5	42/5
CVE-2018-7871	92	49/6	49/6	19/5	53/6
CVE-2021-45078†	2.8	2/2	1/1	1/1	2/2
CVE-2021-20294†	8	1/1	1/1	0	1/1
CVE-2021-20284†	5.2	1/1	1/1	1/1	1/1
CVE-2020-35493†	10.8	5/1	5/1	-	5/1
CVE-2020-16592†	2	0	0	0	0
CVE-2019-20094†	2	0	0	0	0
CVE-2019-20024†	2	0	0	0	0
CVE-2021-3272†	2	0	2/2	0	2/2
CVE-2020-27828	4	1/1	1/1	1/1	3/2
CVE-2018-19543†	2	0	0	0	0
CVE-2018-19540†	3.2	0	0	0	0
CVE-2021-3246	2	1/1	1/1	1/1	1/1
CVE-2020-21676†	21	17/4	12/4	5/2	20/4
CVE-2020-21675†	15.6	10/4	9/4	2/2	11/4
CVE-2023-1916	7.2	3/2	2/2	3/3	4/3
CVE-2023-0799	11.6	19/6	12/3	10/5	22/7
CVE-2023-0804†	15.2	17/4	15/4	2/1	18/4
CVE-2022-3598	2.8	7/4	7/4	1/1	10/5
CVE-2020-19143	2.4	2/1	0	0	2/1
CVE-2019-7663	4.4	5/3	2/2	2/2	5/3
CVE-2023-30084	28.4	36/6	26/5	5/4	41/6
CVE-2023-30083	51.2	54/6	28/5	7/5	57/6
CVE-2021-34341	21.4	23/3	1/1	1/1	24/3
CVE-2021-34339	46	12/3	6/2	2/2	12/3
CVE-2021-34338	30.4	19/4	7/2	3/2	19/4
CVE-2018-9132	82.8	27/5	15/5	14/3	37/5
CVE-2018-7875	88.6	52/5	36/5	16/4	52/5
CVE-2022-47673†	21.2	19/4	9/2	-	19/4
CVE-2022-45703	4.4	4/2	1/1	1/1	4/2
CVE-2022-4285†	6.8	5/4	3/3	1/1	5/4
CVE-2020-35448†	9.6	8/1	6/1	1/1	8/1
CVE-2020-16591†	2.6	1/1	0	0	1/1
CVE-2023-31724	15	21/5	12/4	1/1	23/5
CVE-2023-29582	24	7/3	4/3	1/1	8/3
CVE-2021-33468	8	11/4	6/3	3/2	11/4
CVE-2021-33466	18	25/5	13/3	0	27/5
CVE-2021-33465	14	13/4	6/2	0	13/4
CVE-2022-26981†	6.8	10/1	3/1	0	11/1
CVE-2019-16165	9.2	2/2	1/1	0	2/2
Total Impact		**856**	584	202	941
Increase		+0%	+46.6%	+323.8%	-9.0%

The results show that, in the total of 50 CVEs, Sleuth discovers 856 new bug impacts, increasing by 46.6% and 323.8% respectively, compared to *afl-cexp* and Evocatio. Besides, we calculated the total number of bug impacts found by these three tools, with a total of 921. Sleuth only missed 9.0% of the total impacts. Surprisingly, sleuth successfully identifies new bug impacts of 43 CVEs, covering almost all types of impacts.

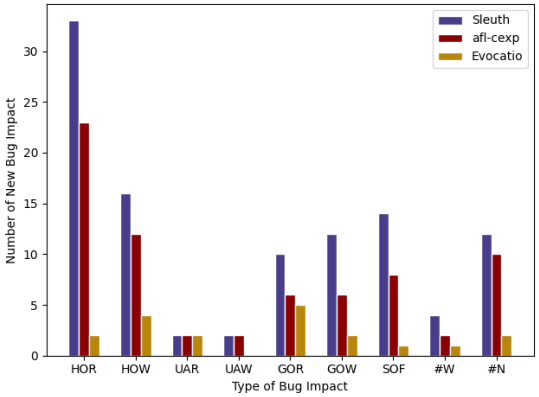
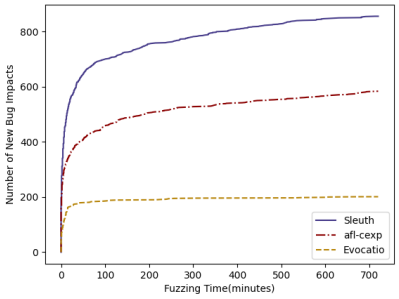
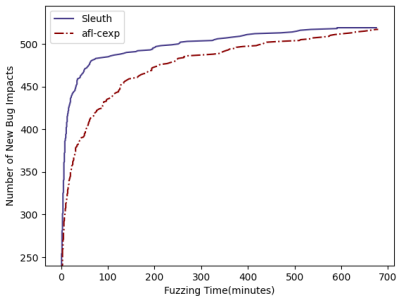


Figure 4: Impacts found on the "Completely Fixed" CVEs.



(a) All Impacts Discovered by Three Fuzzers



(b) Same Impacts Discovered by Sleuth and *afl-cexp*

Figure 5: New Bug Impacts Discovered Over Time

As for the number of switches, we calculate the average number of switches in 5 rounds of testing. Due to our approach priority of in-depth exploration on the initial crash position, fuzzer will initially switch from the traditional exploration to the in-depth exploration mode when the beginning of execution. When the conditions for switching are met during in-depth exploration, breadth exploration will be conducted. Therefore, the number of switchings for each test case is at least 2. Through calculation, Sleuth determined that, in total, the count of switches is approximately 1002, and on average, a new impact requires switching 1.2 times. Due to the switch only made on the seed scheduling strategy, the effect on efficiency is minimal.

Due to false positives generated by static analysis during the pre-analysis phase, the identified impacts may not belong to the same bug. To further illustrate the advancement of Sleuth, we collect 21 "completely fixed" CVEs. That is, for a tested CVE, if all PoCs generated by the three fuzzers do not cause crashes in the patched version for this CVE, we consider this CVE to be "completely fixed". We can confirm that the impacts exposed by these PoCs are caused by the same root cause and belong to the same bug.

We count the overall quantity of new bug impacts with different types for "completely fixed" CVEs in Figure 4. We categorize impact types as described in Table 1. This enables a more intuitive representation of the varying impacts caused by the same bug. The results show that Sleuth can discover 105 new impacts, higher than the 71 of *afl-cexp* and the 19 of Evocatio. Besides, the number of impacts for each type is superior to other fuzzers. In the following statement, we conduct a detailed comparison between Sleuth and other fuzzing tools separately.

6.2.1 Sleuth vs. afl-cexp. In total, Sleuth and *afl-cexp* both find new impacts on most CVEs. However, Sleuth discovers all types of new impacts in 38 CVEs, while *afl-cexp* can only satisfy 21 CVEs. Analysis of test cases revealed that, in the test cases involving a large number of bug impacts, such as CVE-2022-47673 and CVE-2018-9132, multiple impacts still exist behind the discovered crash positions. Although *afl-cexp* discovers adjacent bug impacts by exploring different paths, it misses many bug impacts that require in-depth exploration. On the contrary, Sleuth assigns appropriate energy to regions that have already been executed, thereby uncovering bug impacts situated deep within by changing the data flow. Furthermore, this situation also exists in some CVEs that contain a small number of bug impacts, such as CVE-2020-16591 and CVE-2020-19143. Sleuth can easily discover unknown bug impacts that *afl-cexp* fails to find.

Additionally, in some test cases, Sleuth discovers fewer bugs impacts compared to *afl-cexp*, such as CVE-2021-3272. We suspect that the issue may stem from inaccurate instrumentation of potential bug impacts through static analysis, resulting in the exclusion of some code regions that should been explored. We will consider appropriate methods to optimize this situation in future work.

6.2.2 Sleuth vs. Evocatio. Evocatio is used to explore more new capabilities, and it focuses more on exploring several regions through only changing the data flow. To meet this goal, it restricts the length of the input. Due to the restriction on the mutation length of seeds, Evocatio can only discover a small number of bug impacts. Additionally, as it relies on inferring each byte of the seed, it is unable to process some seeds that are still excessively long after being reduced using *afl-tmin*, such as CVE-2022-47673 and CVE-2020-35493. Sleuth utilizes lightweight feedback-guided mutation without constraining seed length, enabling the exploration of bug impacts in more code regions. Predictably, in certain cases, Evocatio is able to discover subsequent difficult-to-find bug impacts by continuously executing to the same crash position. For example, Evocatio discovered an additional *HOR* impact in CVE-2020-27828, whose

Table 3: The Efficiency of Discovering New Bug Impacts. The "Initial Bug Impact" means the impact exposed by the initial PoC. The "Discovered New Bug Impacts" indicates the impacts newly discovered. The "Time" indicates the first time the new impact was found on each fuzzer. The dash "-" means the corresponding impact is not discovered by the corresponding tool within 12 hours. We sample only some of its newly identified impacts for illustration purposes. We show the complete data in [14].

CVE ID	Initial Bug Impact	Discovered New Bug Impacts	Time(in hours)		
			afl-cexp	Evocatio	Sleuth
CVE-2018-20591	heap-buffer-overflow Read at decompile.c:1843	heap-buffer-overflow Write at decompile.c:115	-	-	2.20
		heap-use-after-free Read at decompile.c:2864	3.24	-	0.08
CVE-2019-16165	heap-use-after-free READ at parser.c:1298	heap-buffer-overflow READ at parser.c:302	-	-	8.56
CVE-2019-7663	wild-address-read at tiffcp.c:1245	heap-buffer-overflow WRITE at tiffcp.c:1245	<0.01	0.19	0.02
CVE-2020-16591	wild-adress Read at readelf.c:12155	global-buffer-overflow Read at readelf.c:12155	-	-	0.02
CVE-2021-20284	heap-buffer-overflow READ at elf.c:12694	heap-use-after-free READ at elf.c:12694	0.01	1.72	<0.01
CVE-2021-3246	heap-buffer-overflow WRITE at ms_adpcm.c:279	heap-buffer-overflow WRITE at ms_adpcm.c:280	0.03	0.01	<0.01
CVE-2022-47673	heap-buffer-overflow Read at vms-alpha.c:4580	heap-buffer-overflow Write at vms-alpha.c:4551	-	-	8.44
CVE-2023-0799	wild-address-read at tiffcrop.c:3701	heap-buffer-overflow Write at tiffcrop.c:3611	-	-	10.52
		heap-use-after-free Write at tiffcrop.c:6487	-	-	3.08
CVE-2022-4285	wild-address-read at elf.c:1969	global-buffer-overflow READ at elf.c:1971	-	1.58	4.33
CVE-2023-31724	wild-adress Read at nasm-pp.c:3563	heap-use-after-free Read at nasm-pp.c:5017	-	-	0.94

initial impact is *HOW*. In practice, Evocatio is more commonly used for bug capability exploration. Sleuth can provide sufficient seeds, assisting Evocatio in covering more bug capabilities.

6.3 RQ2:Efficiency Evaluation

In this section, we conduct a comparative analysis on the time efficiency of different fuzzing tools in discovering new bug impacts. We first select 10 CVEs that are triggered by different binary programs and contain high-risk impacts as the evaluation set. We present the results in Table 3 and provide some necessary explanations. Due to the large number of impacts, we sample only some of the newly identified impacts for illustration purposes. The complete data can be accessed at [14]. The experiment shows that Sleuth is capable of discovering significant bug impacts in a short time.

Figure 5a shows the number of new bug impacts discovered by each fuzzing tool over time. In general, Sleuth is able to identify a substantial number of new bug impacts in less time, and its growth remains consistently stable. The *afl-cexp* also discovers a large number of bug impacts initially, but there is no obvious growth for a long subsequent period. Whereas Evocatio only discovers a small number of bug impacts in the initial phase. To highlight Sleuth's efficiency advantage, we gather the same bug impacts found by Sleuth and *afl-cexp*, and test the trend of the number of discoveries over time. We limit the time for discovering new bug impacts to 700 minutes, and the result (Figure 5b) indicates that Sleuth's performance in efficiency remains exceptional.

The experimental results above have demonstrated the ability of Sleuth to efficiently discover more new bug impacts. As we mentioned in section 2.1, developers can more quickly infer the severity of bugs based on these newly discovered bug impacts.

6.4 RQ3:Patch Testing

Different PoCs generated by Sleuth may trigger the same bug at multiple positions through different control flows and data flows. Developers may not fully consider the patching scope due to using only a single PoC to analyze the root cause. From this perspective, Sleuth should also be able to verify the completeness of bug patches. In response, we design violation testing for PoCs that expose new bug impacts in the patched software version.

In this experiment, we select 34 test cases from our benchmark, which are able to find corresponding patches in the newly submitted development version. In section 6.2, we have detected 21 "completely fixed" CVEs. In the remaining test cases, excluding the 16 unfixed bugs, Sleuth successfully found 13 bugs that were not completely fixed. We list these incompletely patched bugs in Table 4.

Table 4: Incompletely Patched Bug discovered by Sleuth. For each bug we provide the official patch (the commit) and a brief fix measure.

Program	Bug	Patch	Patch Details
Binutils	CVE-2022-45703	69bfd175 [10]	Amend var calculation
Libtiff	CVE-2023-1916	1dff4271 [12]	Add boundary check
Libtiff	CVE-2023-0799	afaabc3e [11]	Amend function parameter
Libtiff	CVE-2022-3598	cfbb883b [9]	Increase buffer size
Libtiff	CVE-2020-19143	84b8b9a3 [6]	Change boundary check
Libtiff	CVE-2019-7663	802d3cbf [5]	Add check before using var
Libming	CVE-2018-7871	3a000c7 [1]	Add boundary check
Libming	CVE-2018-8964	3a000c7 [1]	Add check before using var
Libming	CVE-2018-9009	1d698a4 [2]	Add function to check var
Libming	CVE-2018-9132	d13db01 [3]	Add check before using var
Libming	CVE-2018-7875	3a000c7 [1]	Add boundary check
Libsndfile	CVE-2021-3246	243b518 [8]	Change boundary check
Jasper	CVE-2020-27828	4cd52b5 [7]	Add boundary check

For these incompletely patched bugs, we further analyze the new crashes triggered by PoCs generated by Sleuth. According to the details of the patch and the bug impacts we discovered using Sleuth, we categorize the incompletely patched bugs into three situations: (i) 7 out of 13 incompletely patched bugs are caused by new impact types (CVE-2022-45703, CVE-2023-0799, CVE-2022-3598, CVE-2019-7663, CVE-2018-8964, CVE-2018-9009, CVE-2020-27828). For example, CVE-2023-0799's initial impact is the wild-address-read. Only Sleuth identified HOW and UAW which still exist in the patched version. (ii) three patched bugs still exposed the same impact type as the initial PoC but at different positions

(CVE-2023-1916, CVE-2020-19143, CVE-2021-3246). For example, CVE-2021-3246 initially exposed a GOW at line 279 of ms_adpcm.c. Sleuth discovered another GOW at line 280, which remained after patching. (iii) The remaining three (CVE-2018-7871, CVE-2018-9132, CVE-2018-7875) incompletely patched bugs exhibit the two situations mentioned above.

Most of the incompletely fixed bugs mentioned above have been completely repaired through subsequent updates by developers. Fortunately, we still find that the fix for CVE-2023-1916 was still incomplete. The new impact of this CVE is caused by different memory object within the same memory region, triggering crashes at multiple positions. We submitted new PoCs to the developers in response to the newly identified crash and applied for a corresponding CVE identifier. The bug has been assigned CVE-2023-3164.

7 Related Work

Memory Corruption Bugs originate from illegal access to uninitialized or non-owned memory, as well as incorrect management operations on allocated memory. Existing research [45] has classified memory bugs into spatial bugs (e.g., heap buffer overflow) and temporal bugs (e.g., use-after-free). Fuzzing can trigger crashes by randomly generating seeds, and then utilizing sanitizers [32, 42] to check for memory violations. Many studies explore memory corruption bugs that exist in complex contexts and paths by introducing sensitive coverage metrics [28, 30, 40] and precise seed mutation strategies [16, 21, 27]. Moreover, with the help of static analysis techniques, such as taint analysis [21, 36, 41], symbolic execution [17, 43, 50], etc., the performance of detecting memory corruption bugs using fuzzing has also been improving. Differently, Sleuth focuses on exploring the security impacts that individual memory corruption bugs can produce, rather than aiming to detect more bugs in a program.

Automatic Exploit Generation shares similar goals to Sleuth. This work also involves initially identifying critical bug impacts to bring the bug to an exploitable state. For example, Revery [47] utilizes path concatenation to bypass the original crash and collect new impacts in the program. KOOBE [22] emphasizes different capabilities, such as the size of the vulnerable object, and uncovers hidden impacts using capability-guided fuzzing. Due to the consideration of end-to-end successful exploits in automatic exploit generation, it is necessary to construct specific exploits. Therefore, the exploration only focuses on the specific bug impacts, disregarding diversity. Differently, Sleuth does not carry out exploits, we focus on exploring more common and diverse bug impacts.

Bug Impact Exploration has been utilized to assist with bug patching and automated vulnerability exploitation. It collects more impacts by exploring various contexts and paths toward a target bug. AFL++'s *afl-cexp* [26] explores the crash's state space via code coverage-guided fuzzing and generates new PoCs that induce the same bug. Similarly, based on AFL++, Evocatio [33] uses a novel fuzzer, which can explore more new bug capabilities, including bug impacts. However, without considering the different crash positions for the same bug, they may overlook some unique impacts. Sleuth refers to the design of the aforementioned technique and balances in-depth and breadth exploration to discover more bug impacts.

Kernel-space fuzzing also shares the similar method in exploring bug impacts. SyzScope [54] combines impacts coverage-guided fuzzing, taint analysis, and symbolic execution to identify the security impact of fuzzer-exposed bugs in the kernel. GREBE [38] utilizes static taint analysis and an object-driven kernel fuzzer to explore bug impacts that might have higher exploitation potential than the one shown in the original kernel error report. Indeed, these two techniques consider the in-depth and breadth exploration, but without balancing through dynamic switching. Sleuth is currently implemented on AFL++, which only focuses on user-space programs, and we aim to make Sleuth adapt to a wider range of programs in the real world. We will consider applying our method to kernel fuzzers in the future.

Patch Testing aims to ensure that the newly submitted patch can completely fix the current bug without introducing new bugs [20, 29]. Several studies [34, 48] use PoCs clustering techniques to help developers locate the root cause of bugs. They utilize existing dynamic detection techniques to generate a large number of PoCs, and then group them and regard their commonality within the same group as the root cause. However, they lack the ability to discover new bug impacts, and the proposed patches may only be effective for the existing set of PoCs. In contrast, Sleuth aims to produce a set of PoCs that trigger various bug impacts, which may still trigger bugs in patched programs, helping developers establish correct patches.

8 Conclusion

Software developers can not fully understand the impacts of a bug when only a single PoC provided. They need to further analyze the bug before fixing it, which is time-consuming and labor-intensive. Some studies can identify new bug impacts, but they are plagued by the problem of balancing the time of in-depth and breadth exploration. For this purpose, we design and develop a switchable dual-mode fuzzer, Sleuth, to efficiently discover more bug impacts. Our tool builds upon existing fuzzing tools and has been empirically demonstrated to outperform state-of-the-art techniques. In addition, we demonstrate that Sleuth is effective in assisting bug severity assessment and patch testing.

Data Availability Statement

The artifact [13] for this paper is publicly available at: https://doi.org/10.5281/zenodo.12668172.

Acknowledgment

We thank the anonymous reviewers for their insightful comments and opinions for improving this work. This work is partially supported by the National Natural Science Foundation of China (No. 62172407), and the Youth Innovation Promotion Association CAS.

References

[1] 2018. *CVE-2018-7875, CVE-2018-8964, CVE-2018-7871 Patch.* https://github.com/libming/libming/commit/3a000c7b6fe978dd9925266bb6847709e06dbaa3
[2] 2018. *CVE-2018-9009 Patch.* https://github.com/libming/libming/commit/1d698a4b1f03d6136bbf2b0171b86985be553454
[3] 2018. *CVE-2018-9132 Patch.* https://github.com/hlef/libming/commit/d13db01ea1c416f51043bef7496cfb25c2dde29a
[4] 2019. *ASAN public interface.* https://github.com/google/sanitizers/wiki/AddressSanitizer

[5] 2019. *CVE-2019-7663 Patch.* https://gitlab.com/libtiff/libtiff/-/commit/802d3cbf3043be5dce5317e140ccb1c17a6a2d39

[6] 2020. *CVE-2020-19143 Patch.* https://gitlab.com/libtiff/libtiff/-/commit/84b8b9a3e7c5eca4bb7fea85bbd67d67507946e7

[7] 2020. *CVE-2020-27828 Patch.* https://github.com/jasper-software/jasper/pull/253/commits/4cd52b5daac62b00a0a328451544807ddecf775f

[8] 2021. *CVE-2021-3246 Patch.* https://github.com/libsndfile/libsndfile/pull/713/commits/243b518137d9a921a4c8630accdde2b6b64975b2

[9] 2022. *CVE-2022-3598 Patch.* https://gitlab.com/libtiff/libtiff/-/commit/cfbb883bf6ea7bedcb04177cc4e52d304522fdff

[10] 2022. *CVE-2022-45703 Patch.* https://sourceware.org/git/gitweb.cgi?p=binutils-gdb.git;h=69bfd1759db41c8d369f9dcc98a135c5a5d97299

[11] 2023. *CVE-2023-0799 Patch.* https://gitlab.com/libtiff/libtiff/-/commit/afaabc3e50d4e5d80a94143f7e3c997e7e410f68

[12] 2023. *CVE-2023-1916 Patch.* https://gitlab.com/libtiff/libtiff/-/merge_requests/476

[13] 2024. *Replication package.* https://doi.org/10.5281/zenodo.12668172

[14] 2024. *Sleuth.* https://sites.google.com/view/sleuth-fuzzing-site

[15] Lars Ole Andersen. 1994. Program analysis and specialization for the C programming language. (1994). https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=b7efe971a34a0f2482e0b2520ffb31062dcdde62

[16] Cornelius Aschermann, Sergej Schumilo, Tim Blazytko, Robert Gawlik, and Thorsten Holz. 2019. REDQUEEN: Fuzzing with Input-to-State Correspondence.. In *NDSS*, Vol. 19. 1–15. https://doi.org/10.14722/ndss.2019.23371

[17] Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. 2018. A survey of symbolic execution techniques. *ACM Computing Surveys (CSUR)* 51, 3 (2018), 1–39. https://doi.org/10.1145/3182657

[18] Tim Blazytko, Moritz Schlögel, Cornelius Aschermann, Ali Abbasi, Joel Frank, Simon Wörner, and Thorsten Holz. 2020. {AURORA}: Statistical crash analysis for automated root cause explanation. In *29th USENIX Security Symposium (USENIX Security 20)*. 235–252. https://www.usenix.org/conference/usenixsecurity20/presentation/blazytko

[19] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2329–2344. https://doi.org/10.1145/3133956.3134020

[20] Lingchao Chen, Yicheng Ouyang, and Lingming Zhang. 2021. Fast and precise on-the-fly patch validation for all. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1123–1134. https://doi.org/10.1109/ICSE43902.2021.00104

[21] Peng Chen and Hao Chen. 2018. Angora: Efficient fuzzing by principled search. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 711–725. https://doi.org/10.1109/SP.2018.00046

[22] Weiteng Chen, Xiaochen Zou, Guoren Li, and Zhiyun Qian. 2020. {KOOBE}: towards facilitating exploit generation of kernel {Out-Of-Bounds} write vulnerabilities. In *29th USENIX Security Symposium (USENIX Security 20)*. 1093–1110. https://www.usenix.org/conference/usenixsecurity20/presentation/chen-weiteng

[23] Zhengjie Du, Yuekang Li, Yang Liu, and Bing Mao. 2022. WindRanger: a directed greybox fuzzer driven by deviation basic blocks. In *Proceedings of the 44th International Conference on Software Engineering*. 2440–2451. https://doi.org/10.1145/3510003.3510197

[24] Thomas Dullien. 2017. Weird machines, exploitability, and provable unexploitability. *IEEE Transactions on Emerging Topics in Computing* 8, 2 (2017), 391–403. https://doi.org/10.1109/TETC.2017.2785299

[25] Thomas Dullien and Halvar Flake. 2011. Exploitation and state machines. *Proceedings of Infiltrate* (2011). https://downloads.immunityinc.com/infiltrate-archives/Fundamentals_of_exploitation_revisited.pdf

[26] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. {AFL++}: Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. https://www.usenix.org/conference/woot20/presentation/fioraldi

[27] Shuitao Gan, Chao Zhang, Peng Chen, Bodong Zhao, Xiaojun Qin, Dong Wu, and Zuoning Chen. 2020. {GREYONE}: Data flow sensitive fuzzing. In *29th USENIX security symposium (USENIX Security 20)*. 2577–2594. https://www.usenix.org/conference/usenixsecurity20/presentation/gan

[28] Shuitao Gan, Chao Zhang, Xiaojun Qin, Xuwen Tu, Kang Li, Zhongyu Pei, and Zuoning Chen. 2018. Collafl: Path sensitive fuzzing. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 679–696. https://doi.org/10.1109/SP.2018.00040

[29] Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2018. Automatic software repair: A survey. In *Proceedings of the 40th International Conference on Software Engineering*. 1219–1219. https://doi.org/10.1145/3180155.3182526

[30] Adrian Herrera, Mathias Payer, and Antony L Hosking. 2022. DatAFLow: Toward a Data-Flow-Guided Fuzzer. *ACM Transactions on Software Engineering and Methodology* (2022). https://doi.org/10.1145/3587156

[31] Heqing Huang, Yiyuan Guo, Qingkai Shi, Peisen Yao, Rongxin Wu, and Charles Zhang. 2022. Beacon: Directed grey-box fuzzing with provable path pruning. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 36–50. https://doi.org/10.1109/SP46214.2022.9833751

[32] Yuseok Jeon, WookHyun Han, Nathan Burow, and Mathias Payer. 2020. {FuZZan}: Efficient sanitizer metadata design for fuzzing. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. 249–263. https://www.usenix.org/conference/atc20/presentation/jeon

[33] Zhiyuan Jiang, Shuitao Gan, Adrian Herrera, Flavio Toffalini, Lucio Romerio, Chaojing Tang, Manuel Egele, Chao Zhang, and Mathias Payer. 2022. Evocatio: Conjuring Bug Capabilities from a Single PoC. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1599–1613. https://doi.org/10.1145/3548606.3560575

[34] Zhiyuan Jiang, Xiyue Jiang, Ahmad Hazimeh, Chaojing Tang, Chao Zhang, and Mathias Payer. 2021. Igor: Crash deduplication through root-cause clustering. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 3318–3336. https://doi.org/10.1145/3460120.3485364

[35] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.* IEEE, 75–86. https://doi.org/10.1109/CGO.2004.1281665

[36] Caroline Lemieux and Koushik Sen. 2018. Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 475–485. https://doi.org/10.1145/3238147.3238176

[37] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2021. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing* 19, 4 (2021), 2244–2258. https://doi.org/10.1109/TDSC.2021.3051525

[38] Zhenpeng Lin, Yueqi Chen, Yuhang Wu, Dongliang Mu, Chensheng Yu, Xinyu Xing, and Kang Li. 2022. GREBE: Unveiling exploitation potential for Linux kernel bugs. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2078–2095. https://doi.org/10.1109/SP46214.2022.9833683

[39] Antonio Nappa, Richard Johnson, Leyla Bilge, Juan Caballero, and Tudor Dumitras. 2015. The attack of the clones: A study of the impact of shared code on vulnerability patching. In *2015 IEEE symposium on security and privacy*. IEEE, 692–708. https://doi.org/10.1109/SP.2015.48

[40] Sebastian Österlund, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2020. {ParmeSan}: Sanitizer-guided greybox fuzzing. In *29th USENIX Security Symposium (USENIX Security 20)*. 2289–2306. https://www.usenix.org/conference/usenixsecurity20/presentation/osterlund

[41] Sanjay Rawat, Vivek Jain, Ashish Kumar, Lucian Cojocar, Cristiano Giuffrida, and Herbert Bos. 2017. VUzzer: Application-aware Evolutionary Fuzzing.. In *NDSS*, Vol. 17. 1–14. https://doi.org/10.14722/ndss.2017.23404

[42] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. {AddressSanitizer}: A fast address sanity checker. In *2012 USENIX annual technical conference (USENIX ATC 12)*. 309–318. https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany

[43] Nick Stephens, John Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2016. Driller: Augmenting fuzzing through selective symbolic execution.. In *NDSS*, Vol. 16. 1–16. https://doi.org/10.14722/ndss.2016.23368

[44] Yulei Sui and Jingling Xue. 2016. SVF: interprocedural static value-flow analysis in LLVM. In *Proceedings of the 25th international conference on compiler construction*. 265–266. https://doi.org/10.1145/2892208.2892235

[45] Laszlo Szekeres, Mathias Payer, Tao Wei, and Dawn Song. 2013. Sok: Eternal war in memory. In *2013 IEEE Symposium on Security and Privacy*. IEEE, 48–62. https://doi.org/10.1109/SP.2013.13

[46] Julien Vanegue. 2013. The automated exploitation grand challenge. In *H2HC Conference*. https://openwall.info/wiki/_media/people/jvanegue/files/aegc_vanegue.pdf

[47] Yan Wang, Chao Zhang, Xiaobo Xiang, Zixuan Zhao, Wenjie Li, Xiaorui Gong, Bingchang Liu, Kaixiang Chen, and Wei Zou. 2018. Revery: From proof-of-concept to exploitable. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1914–1927. https://doi.org/10.1145/3243734.3243847

[48] Carter Yagemann, Matthew Pruett, Simon P Chung, Kennon Bittick, Brendan Saltaformaggio, and Wenke Lee. 2021. {ARCUS}: symbolic root cause analysis of exploits in production systems. In *30th USENIX Security Symposium (USENIX Security 21)*. 1989–2006. https://www.usenix.org/conference/usenixsecurity21/presentation/yagemann

[49] Wei You, Peiyuan Zong, Kai Chen, XiaoFeng Wang, Xiaojing Liao, Pan Bian, and Bin Liang. 2017. Semfuzz: Semantics-based automatic generation of proof-of-concept exploits. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2139–2154. https://doi.org/10.1145/3133956.3134085

[50] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, and Taesoo Kim. 2018. {QSYM}: A practical concolic execution engine tailored for hybrid fuzzing. In *27th USENIX Security Symposium (USENIX Security 18)*. 745–761. https://www.usenix.org/conference/usenixsecurity18/presentation/yun

[51] Michal Zalewski. 2018. Afl-fuzz: Crash exploration mode. https://afl-1.readthedocs.io/en/latest/fuzzing.html

[52] Michal Zalewski. 2019. american fuzzy lop (2.52 b). *Retrieved April* 10 (2019), 2020. https://lcamtuf.coredump.cx/afl/

[53] Yuchen Zhang, Chengbin Pang, Georgios Portokalidis, Nikos Triandopoulos, and Jun Xu. 2022. Debloating address sanitizer. In *31st USENIX Security Symposium (USENIX Security 22)*. 4345–4363. https://www.usenix.org/conference/usenixsecurity22/presentation/zhang-yuchen

[54] Xiaochen Zou, Guoren Li, Weiteng Chen, Hang Zhang, and Zhiyun Qian. 2022. {SyzScope}: Revealing {High-Risk} Security Impacts of {Fuzzer-Exposed} Bugs in Linux kernel. In *31st USENIX Security Symposium (USENIX Security 22)*. 3201–3217. https://www.usenix.org/conference/usenixsecurity22/presentation/zou

Received 2024-04-12; accepted 2024-07-03